



Penerapan Topologi Mesh Pada *Master-Slave* Untuk Pengiriman Paket Data Sensor Soil Moisture

^{[1]*}Awal Cahyo Bayu Aji, ^[2]Ikhwan Ruslianto

^{[1][2]} Jurusan Rekayasa Sistem Komputer, Fakultas Matematika dan Ilmu Pengetahuan Alam,
Universitas Tanjungpura, Kota Pontianak, Indonesia

e-mail: ^{[1]*}h1051201030@student.untan.ac.id, ^[2]ikhwanruslianto@siskom.untan.ac.id

Email Penulis Korespondensi: h1051201030@student.untan.ac.id

Submitted: 11/09/2025; Accepted: 26/09/2025; Published: 06/04/2026

Abstrak

Sektor pertanian memerlukan sistem pemantauan kelembaban tanah yang andal untuk menunjang pertumbuhan tanaman secara optimal. Penelitian ini merancang serta mengimplementasikan sistem pemantauan kelembaban tanah berbasis arsitektur master-slave menggunakan empat NodeMCU ESP32 dengan penerapan topologi mesh. Topologi mesh tidak dimanfaatkan sepenuhnya untuk pertukaran data, melainkan difokuskan pada mekanisme failover, yaitu ketika node master utama mengalami kegagalan, peran master dialihkan secara statis kepada slave sesuai urutan prioritas. Sistem terdiri atas satu master dan tiga slave, di mana setiap slave terhubung dengan empat sensor soil moisture. Data hasil pembacaan sensor dikumpulkan terlebih dahulu pada masing-masing slave menggunakan metode batch processing, kemudian dikirimkan ke master untuk diteruskan ke cloud server. Mekanisme failover ditetapkan secara statis, sehingga Slave 1 berfungsi sebagai backup master pertama, dilanjutkan oleh Slave 2, dan seterusnya. Pengujian dilakukan melalui sembilan skenario berbeda untuk mengevaluasi kinerja sistem. Hasil menunjukkan pada skenario 1, 2, 3, 4, 7, dan 8 sistem mencapai packet loss 0%, yang termasuk kategori sangat bagus menurut standar TIPHON. Total packet loss keseluruhan sebesar 0,13% juga masih dalam kategori sangat bagus. Dengan demikian, sistem terbukti efektif menjaga keberlanjutan pengiriman data melalui penerapan topologi mesh berbasis failover pada pemantauan pertanian secara real-time.

Kata kunci: IoT, topologi mesh, master-slave, failover, soil moisture monitoring

1. PENDAHULUAN

Sektor pertanian memiliki peran strategis dalam pembangunan nasional Indonesia sebagai negara agraris, tidak hanya sebagai sumber pangan utama, tetapi juga sebagai penyedia bahan baku industri dan sumber energi alternatif. Kontribusi sektor pertanian terhadap Produk Domestik Bruto (PDB) dan penyerapan tenaga kerja menjadikannya pilar ekonomi yang fundamental. Namun, produktivitas pertanian Indonesia masih menghadapi tantangan signifikan, terutama terkait dengan pengelolaan lahan yang belum optimal dan keterbatasan teknologi dalam monitoring kondisi tanah [1].

Parameter fisik tanah, khususnya kelembaban, merupakan faktor kritis yang

mempengaruhi pertumbuhan dan produktivitas tanaman. Kondisi kelembaban tanah yang optimal berkisar antara 50-70% dengan variasi suhu 20-30°C, bergantung pada jenis dan fase pertumbuhan tanaman. Perubahan parameter ini dapat berdampak langsung terhadap efisiensi fotosintesis, absorpsi nutrisi, dan sistem perakaran tanaman. Oleh karena itu, sistem monitoring berkelanjutan terhadap kondisi edafik menjadi kebutuhan dalam implementasi pertanian presisi [2,3].

Implementasi sistem monitoring berbasis Internet of Things (IoT) dengan arsitektur master-slave menghadapi tantangan utama berupa packet loss dan ketidakstabilan transmisi data. Kondisi ini diperparah ketika terjadi kegagalan pada node master yang dapat



menginterupsi seluruh proses akuisisi data. Solusi yang dapat diterapkan adalah implementasi topologi mesh pada arsitektur *master-slave* yang memungkinkan fitur *self-healing* melalui mekanisme *failover* otomatis [4]. Ketika node *master* mengalami kegagalan, sistem dapat secara dinamis menunjuk node *slave* sebagai *backup master* untuk memastikan kontinuitas pengiriman data.

Penelitian terkait yang pernah dilakukan dengan judul "*Visual Self-healing Modelling for Reliable Internet-of-Things Systems*" [5]. Pada penelitian ini membahas tentang pengembangan sistem IoT yang dapat melakukan *self healing* ketika terjadi masalah. Menggunakan Node-RED (platform visual untuk IoT), para peneliti membuat ekstensi khusus yang memungkinkan sistem mendeteksi kerusakan dan langsung mengatasinya secara otomatis. Pengujian dilakukan pada *SmartLab* dengan tiga skenario, ketidak tersediaan message broker, sensor rusak, dan gangguan koneksi. Hasilnya menunjukkan sistem berhasil mendeteksi masalah dan melakukan perbaikan sendiri tanpa bantuan manusia, sehingga sistem IoT menjadi lebih andal dan tidak mudah rusak. Namun, pada penelitian tersebut tidak mengimplementasikan topologi *mesh* pada arsitektur *master-slave* secara spesifik untuk aplikasi monitoring pertanian.

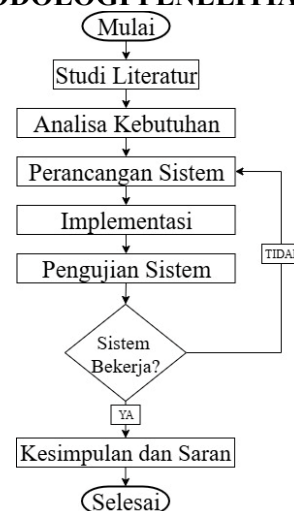
Penelitian terkait yang pernah dilakukan dengan judul "*Development of NRF24L01 Communication Module-Based Temperature Monitoring System*" [6]. Pada penelitian ini membahas tentang pengembangan sistem monitoring suhu berbasis modul komunikasi NRF24L01 dengan arsitektur *master-slave*. Pengambilan data dilakukan dengan mengukur suhu pada rentang 27-50°C melalui sensor PT100 pada Modul Plant. Data dikirim dari Arduino ke LabVIEW secara nirkabel menggunakan NRF24L01. Hasil dari penelitian ini sistem berhasil mengirimkan dan memantau data suhu secara nirkabel dari *slave* ke *master* dengan tingkat akurasi yang baik dan rata-rata persentase error sistem adalah 0,00453%. Namun, pada penelitian tersebut tidak mengintegrasikan topologi *mesh* dan tidak dilengkapi dengan mekanisme *self-healing* untuk mengatasi kegagalan node *master*.

Penelitian terkait yang pernah dilakukan dengan judul "Rancang Bangun Sistem Monitoring Kualitas Tanah Untuk Tanaman

Cabai Berbasis IoT (*Internet of Things*)" [7]. Pada penelitian ini membahas tentang sistem monitoring kualitas tanah untuk tanaman cabai berbasis *Internet of Things* (IoT) menggunakan modul nodeMCU ESP8266, Arduino Uno, serta sensor pH tanah, sensor kelembaban tanah (*soil moisture*), dan sensor suhu dan kelembaban (DHT11). Hasil penelitian dapat mengukur kadar pH tanah dan kelembaban tanah, sehingga dapat membantu petani cabai dalam pemantauan tanaman cabai. Namun, pada penelitian tersebut tidak menggunakan topologi *mesh* dan tidak memiliki fitur *self-healing* untuk memastikan kontinuitas pengiriman data ketika terjadi gangguan pada sistem.

Berdasarkan penelitian yang telah dilakukan, maka diangkat penelitian dengan judul "Penerapan Topologi *Mesh* Pada *Master-Slave* Untuk Pengiriman Paket Data *Sensor Soil Moisture*". Penelitian ini bertujuan untuk mengoptimalkan pengiriman paket data sensor *soil moisture* dengan menerapkan topologi *mesh* pada arsitektur *master-slave* yang dilengkapi dengan mekanisme *self-healing*. Hal ini diharapkan dapat memberikan pengiriman data yang lebih andal dan kontinyu dengan meminimalkan *packet loss*, serta memastikan keberlanjutan transmisi data sensor ke *cloud server* meskipun terjadi kegagalan pada node *master*.

2. METODOLOGI PENELITIAN



Gambar 1 Metode Penelitian

Metode penelitian ini dimulai dengan dengan studi literatur mengumpulkan informasi terkait dengan penerapan topologi *mesh* pada *master-slave* pada ESP32, sensor *soil moisture*, serta parameter perhitungan seperti *packet loss*.

Tahapan selanjutnya adalah melakukan analisis kebutuhan, yang meliputi komponen perangkat keras seperti ESP32 dan sensor kelembaban tanah, serta perangkat lunak seperti Laravel dan Arduino IDE. Analisis ini juga mencakup temuan dari studi literatur yang mendukung pemahaman terhadap kebutuhan sistem yaitu. Pertama ESP32 merupakan mikrokontroler dengan konsumsi daya rendah yang terintegrasi dengan modul komunikasi nirkabel Wi-Fi dan Bluetooth, yang dapat beroperasi secara simultan [8]. Dalam penelitian ini, NodeMCU ESP32 dimanfaatkan sebagai unit pengendali utama yang mengkoordinasikan komunikasi antara node *master* dan tiga node *slave* lainnya yang juga berbasis NodeMCU ESP32. Setiap node *slave* terhubung dengan sensor kelembaban tanah untuk melakukan akuisisi data. Selain itu, NodeMCU ESP32 juga berfungsi sebagai media pengirim data ke *cloud server* untuk keperluan pemantauan secara real-time. Adapun gambar ESP32 dapat dilihat pada Gambar 2.



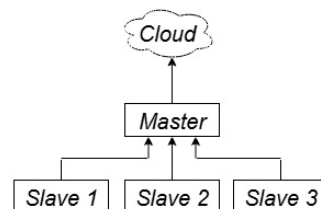
Gambar 2 ESP32

Kedua Sensor *Soil moisture* merupakan perangkat yang berfungsi untuk mengukur kadar air atau tingkat kelembaban yang terkandung di dalam tanah, dan dapat diintegrasikan dengan mikrokontroler untuk pemrosesan data secara digital [9]. Dalam penelitian ini, sensor kelembaban tanah digunakan untuk mendeteksi kondisi kadar air pada media tanam, yang kemudian data dikumpulkan oleh node *slave* dan dikirimkan ke node *master* diteruskan ke *cloud server* untuk keperluan pemantauan dan analisis. Adapun gambar Sensor *Soil moisture* dapat dilihat pada Gambar 3.



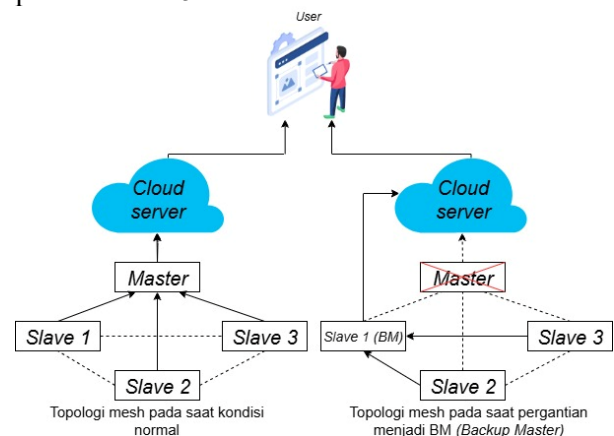
Gambar 3 Sensor *Soil Moisture*

Ketiga *Master-slave*, Dalam arsitektur multiprosesor *master-slave*, prosesor *master* bertindak sebagai pengendali utama yang menginisiasi komunikasi, sedangkan prosesor *slave* hanya merespons perintah dari *master*[10]. *Master* memiliki kendali penuh terhadap operasi dan fungsi *slave*, baik dalam komunikasi perangkat keras maupun proses komputasi. Pada penelitian ini, satu NodeMCU ESP32 berperan sebagai *master* yang mengatur komunikasi dan pengumpulan data dari tiga node *slave* yang terhubung dengan sensor kelembaban tanah. Data yang terkumpul dikirim ke *cloud server* untuk pemantauan dan analisis lebih lanjut.. Adapun arsitektur *master-slave* bisa dilihat pada Gambar 4.



Gambar 4 Arsitektur *Master-Slave*

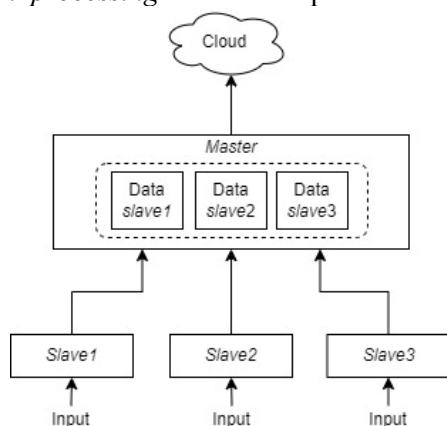
Keempat topologi *mesh* arsitektur jaringan dimana setiap node saling terhubung langsung, memungkinkan komunikasi dua arah tanpa bergantung pada satu jalur pusat [11]. Dalam penelitian ini, topologi *mesh* digunakan untuk membangun jaringan WLAN yang adaptif dan andal. Setiap node dapat meneruskan data dari node lain, menjaga keberlanjutan jaringan saat terjadi kegagalan. Jika *master* tidak berfungsi, *slave* secara otomatis mengambil alih peran secara statis, mengumpulkan data, dan mengirimkannya ke *cloud server*. Adapun topologi *mesh* bisa dilihat pada Gambar 5.



Gambar 5 Topologi *Mesh*

Kelima Konsep *batch processing* tidak dijalankan melalui *Apache Spark*, melainkan

diimplementasikan secara langsung pada ESP32 dengan mengadopsi prinsip dasarnya. Data dari beberapa *slave* dikumpulkan oleh node *master* dalam interval tertentu, kemudian diproses secara kolektif sebelum diteruskan, sehingga menghasilkan wawasan yang lebih mendalam terhadap sistem yang dipantau. [12]. Dalam penelitian ini, node *master* mengumpulkan data dari beberapa node *slave*, menggabungkannya dalam satu batch, dan mengirimkannya ke *cloud server* untuk disimpan dan dianalisis lebih lanjut. Adapun *batch processing* bisa dilihat pada Gambar 6.



Gambar 6 Batch Processing

Keenam *cloud server* adalah *server* virtual yang berjalan di lingkungan komputasi awan dengan dukungan infrastruktur virtualisasi dari penyedia layanan cloud[13]. *Cloud server* ini berfungsi seperti server fisik, namun lebih fleksibel dalam konfigurasi dan pengelolaan beban kerja. Dalam penelitian ini, *cloud server* digunakan untuk menyimpan dan mengakses data sensor kelembaban tanah secara real-time dari mana saja dan kapan saja.

Ketujuh Arduino IDE adalah perangkat lunak pemrograman mikrokontroler berbasis C/C++ yang menyediakan antarmuka sederhana serta pustaka bawaan untuk mempermudah penulisan dan pengunggahan kode melalui USB [14]. Kemudahan antarmuka dan ketersediaan pustaka membuat proses pengembangan dan pengujian menjadi lebih efisien, khususnya dalam pengembangan sistem berbasis mikrokontroler.

Kedelapan Laravel adalah framework open-source berbasis PHP yang menerapkan arsitektur Model-View-Controller (MVC) untuk memudahkan pengelolaan rute, interaksi basis data, dan pembuatan API [15]. Dalam penelitian ini, Laravel digunakan untuk

mengembangkan aplikasi web sistem pemantauan kelembaban tanah, dengan dukungan fitur keamanan seperti perlindungan CSRF, validasi input, autentikasi, dan otorisasi.

Kesembilan *wireshark* merupakan alat analisis jaringan yang digunakan untuk menangkap dan menampilkan data paket secara detail dalam waktu nyata, sehingga efektif untuk memantau dan menganalisis aktivitas jaringan [16]. Dalam penelitian ini, *Wireshark* dimanfaatkan untuk menganalisis waktu failover antar node pada topologi mesh *master-slave*. Saat terjadi *failover*, peran *master* berpindah ke node cadangan, dan pengiriman data dari *slave* terhenti sementara. Melalui analisis timestamp pada paket yang diterima, *Wireshark* digunakan untuk mengukur jeda waktu antara paket terakhir dari node lama dan paket pertama dari node baru.

Kesepuluh *packet loss* merupakan parameter yang mengukur jumlah paket data yang gagal dikirimkan dalam proses transmisi jaringan (TIPHON, 2002). Kehilangan paket dapat disebabkan oleh berbagai faktor, seperti kelebihan beban jaringan, kerusakan perangkat keras, interferensi sinyal, hambatan fisik, atau kesalahan konfigurasi. Kondisi ini berdampak pada penurunan kinerja jaringan dan ketidakstabilan aplikasi yang bergantung pada transmisi data. Nilai *packet loss* dapat dihitung menggunakan persamaan yang ditunjukkan pada persamaan 1 [17].

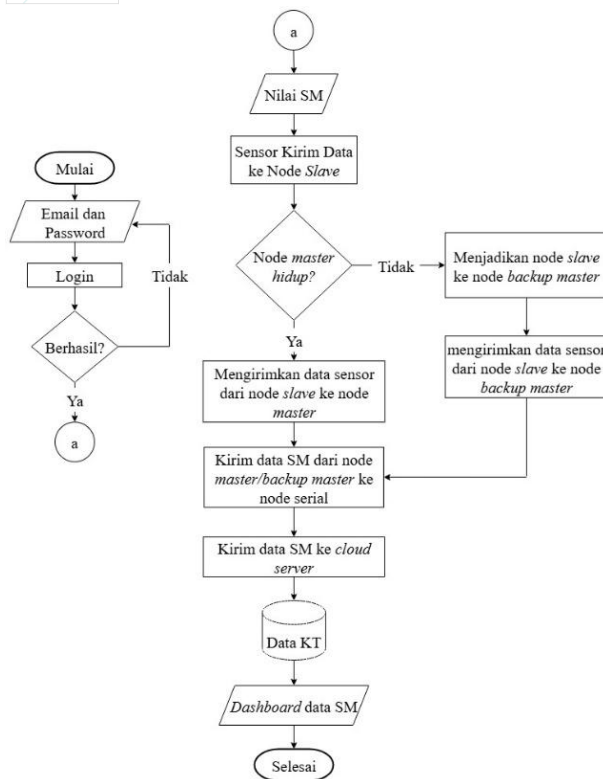
$$Packet\ loss = \frac{paket\ dikirim - paket\ diterima}{paket\ data\ dikirim} \times 100\% \quad (1)$$

Untuk mengevaluasi kualitas throughput diperlukan suatu standar menggunakan TIPHON yang dapat dilihat pada Tabel 1.

Tabel 1 Kategori *Packet Loss* [17]

Kategori <i>Packet loss</i>	<i>Packet loss</i> (%)	Indeks
Sangat Bagus	0 - 3%	4
Bagus	3 - 15%	3
Sedang	15 - 25%	2
Jelek	>25%	1

Setelah kebutuhan sistem terdefinisi, tahap selanjutnya adalah merancang sistem yang menggunakan ESP32 sebagai mikrokontroler utama dengan menerapkan topologi *mesh* pada *master-slave* untuk pergantian antar node. Adapun diagram alir untuk pergantian node ketika *master* mengalami kegagalan bisa dilihat pada Gambar 7.



Gambar 7 Diagram Alir Proses Pengalihan Tugas ke Slave Saat Node Master Mati

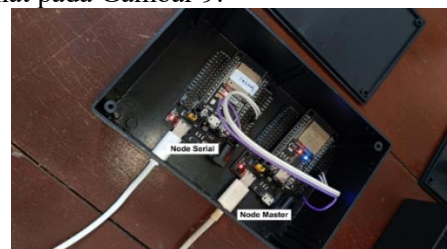
Implementasi sistem *monitoring* yang dilakukan pada topologi *mesh* dengan *master-slave* untuk memastikan berjalan dengan baik pada saat node *master* mati. Sistem ini menggunakan Tiga node *slave* masing-masing dilengkapi dengan empat sensor *soil moisture* yang terhubung ke mikrokontroler ESP32 untuk mengukur kadar kelembaban tanah secara berkelanjutan. Data yang diperoleh diproses oleh ESP32 pada setiap node dan dikirimkan secara nirkabel ke node *master* dalam jadwal terkoordinasi guna menghindari interferensi dan memastikan data yang akurat dan terkini. Sistem ini juga menerapkan mekanisme failover dengan menetapkan salah satu node *slave* sebagai *backup master* yang akan mengambil alih fungsi koordinasi jika *master* utama gagal. Namun, karena keterbatasan topologi *mesh* pada ESP32 di mana modul *Wi-Fi* tidak dapat digunakan secara bersamaan untuk komunikasi *mesh* dan koneksi internet maka *backup master* tidak dapat mengirim data langsung ke *cloud server*. Oleh karena itu, digunakan node *serial* sebagai penghubung melalui koneksi fisik RX dan TX untuk meneruskan data ke *cloud server*. Pendekatan ini memastikan sistem pemantauan tetap berjalan andal meskipun terjadi kegagalan pada

node *master* utama. Proses implementasi sistem pada setiap node *slave* ditampilkan melalui pada Gambar 8.



Gambar 8 Implementasi pada node *slave*

Pada node *master* utama digunakan untuk mengumpulkan data dari node *slave* terlebih dahulu. Selain itu, node *serial* berfungsi sebagai perantara untuk mengirimkan data yang telah dikumpulkan oleh node *master*. Hal ini diperlukan karena ketika node *master* utama sedang menerima data sensor menggunakan topologi *mesh*, node tersebut tidak bisa mengirimkan data ke *server*. Untuk mengatasi pada masalah ini, node *serial* digunakan dengan menghubungkan node *master* melalui kabel ke pin RX dan TX. Implementasi node *master* bisa dilihat pada Gambar 9.



Gambar 9 Implementasi Sistem pada Node *master*

Sistem komunikasi antara node *master* dan *slave* menggunakan topologi *mesh* dirancang untuk memungkinkan pengiriman data sensor secara nirkabel tanpa bergantung pada *Wi-Fi* atau internet. Dalam topologi ini, setiap ESP32 dapat bertindak sebagai pengirim, penerima, atau perantara (relay), mendukung komunikasi langsung antar-node. Keunggulan utamanya adalah sifat terdistribusi dan kemampuan *self-healing*, yang menjaga konektivitas meskipun terjadi kegagalan pada salah satu node. Dengan demikian, sistem memastikan pengiriman data sensor yang andal

dan berkelanjutan, sebagaimana ditunjukkan pada Gambar 10.



Gambar 10 Rangkaian Implementasi Node Master dan Node Slave dengan Sensor Kelembaban Tanah

Aplikasi berbasis website digunakan untuk membantu pemantauan sistem *monitoring* tanah, aplikasi ini dirancang untuk menyajikan informasi secara *real-time* data kelembaban tanah. Selain itu, *packet loss* juga ditampilkan untuk memberikan gambaran menyeluruh tentang kinerja jaringan dan stabilitas sistem. Dengan informasi ini, dapat memastikan kondisi lingkungan tanaman tetap optimal. *dashboard web* untuk sistem pemantauan kelembaban tanah Gambar 11.



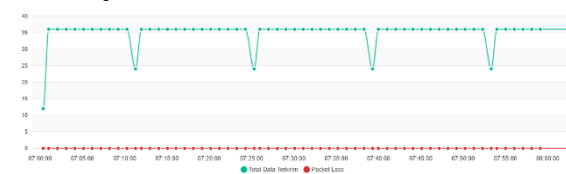
Gambar 11 Dashboard Web

Untuk Sistem Pemantauan Kelembaban Tanah Tahap terakhir adalah pengujian sistem untuk mengevaluasi kinerja pergantian antar node dengan fokus parameter *packet loss*. Hasil pengujian ini menjadi dasar untuk mendapatkan kesimpulan dari penelitian ini.

3. HASIL DAN PEMBAHASAN

Pada Skema pengujian pada Sistem Monitoring Tanah Berbasis *Master-Slave* Selama satu jam pengujian, node *master* menerima 2.088 data dari tiga node *slave* dengan 0% *packet loss*, yang menurut standar TIPHON dikategorikan *sangat bagus*, menunjukkan komunikasi yang andal. Ukuran total data mencapai 8.352 byte, mencerminkan proses *batching* dari seluruh *slave*. Masing-masing node *slave* mengirimkan 696 data dalam periode yang sama, juga dengan 0% *packet loss* dan ukuran data per node mencapai 2.800 byte. Grafik pengujian pada sistem monitoring tanah

berbasis *master-slave* bisa dilihat pada Gambar 12 untuk *packet loss* dan Gambar 13 untuk ukuran paket data.



Gambar 12 Grafik Jumlah Pengiriman dan Packet loss Berbasis Master-Slave pada Node (Master, Slave 1, 2, Dan 3)



Gambar 13 Grafik Ukuran Rata-rata Paket Data Berbasis Master-Slave pada Node (Master, Slave 1, 2, Dan 3)

Selanjutnya pada pengujian pada Sistem Monitoring Tanah berbasis *master-slave* untuk pergantian dari *master* ke *slave* dengan topologi *mesh* dilakukan selama tiga hari. dengan interval pengiriman data setiap 20 detik. Pada hari pertama (29 April 2025), dilakukan 9 skenario pengujian.

Node *master* berhasil mengirimkan 4.164 data dengan *packet loss* 0%, sementara setiap *slave* mengirimkan rata-rata 696 data. *Slave 1* berperan sebagai *backup master* pada skenario 2, 6, 8, dan 9, serta sebagai *slave* biasa pada skenario 1 dan 7, dengan total 4.204 data dan *packet loss* 0%. *Slave 2* mengirimkan 4.888 data, menjadi *backup master* pada skenario 3 dan 5, serta sebagai *slave* biasa di skenario lain dengan *packet loss* sebesar 0,16%. Sementara itu, *Slave 3* mengirimkan total 5.596 data dan menjadi *backup master* pada skenario 4. Di skenario lain, ia bertindak sebagai *slave* biasa dengan *packet loss* sebesar 0,29%.

Berdasarkan Gambar 14–17, node *master* (Gambar 14) memperlihatkan grafik yang konsisten tanpa adanya *packet loss*, sehingga dapat disimpulkan proses pengumpulan data dari seluruh *slave* berjalan stabil. Pada *slave 1* (Gambar 15) dan *Slave 2* (Gambar 16), terlihat adanya *packet loss* kecil yang muncul ketika keduanya beralih peran menjadi *backup master*, namun nilainya tetap di bawah 1% sehingga tidak mengganggu keseluruhan kinerja sistem. Sementara itu, *slave 3* (Gambar 17) menunjukkan penurunan tajam ketika transisi menjadi *backup master*, tetapi grafik kembali

normal setelah proses *failover* selesai. Hal ini menegaskan bahwa *packet loss* umumnya hanya muncul pada saat peralihan peran node, sementara setelah itu sistem mampu menjaga kestabilan komunikasi dengan baik.

Secara keseluruhan, sistem berhasil mengirimkan total 18.852 data dengan *packet loss* rendah sebesar 0,13%. Menurut standar TIPHON, hal ini masuk dalam kategori “sangat bagus”. *Packet loss* umumnya terjadi saat transisi peran, ketika node pengganti baru aktif setelah node sebelumnya masih sempat mengirimkan data. Namun, sistem kembali normal setelah proses *failover*, terbukti dengan pengiriman data yang konsisten. Adapun grafik *packet loss* di masing-*slave* bisa dilihat pada Gambar 14,15,16,17.



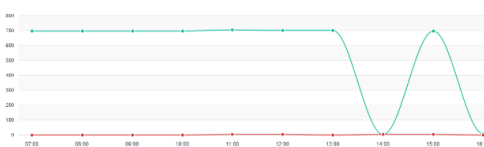
Gambar 14 Grafik *Packet loss* Pada Node Master



Gambar 15 Grafik *Packet loss* Pada Node Slave Satu



Gambar 16 Grafik *Packet loss* Pada Node Slave Dua



Gambar 17 Grafik *Packet loss* Pada Node Slave Tiga

Adapun Tabel 3 skenario untuk melihat hasil secara keseluruhan, bisa dilihat pada tabel 2.

Tabel 2 Pengujian Sembilan Skenario

skenario	Slave 1 (Jumlah data)	Slave 2 (Jumlah data)	Slave 3 (Jumlah data)	Jumlah Total data	Total byte data	Total packet loss	Data Terakhir Terkirim	waktu (ms)
Skenario 1 master ambil alih	696	696	696	2.088	8.352	0%	0	0
Skenario 2 slave 1 ambil alih	700	696	696	2.092	8.368	0%	0	31.376
Skenario 3 slave 2 ambil alih	0	700	696	1.396	5.584	0%	0	35.630
Skenario 4 slave 3 ambil alih	0	0	696	696	2.784	0%	0	40.593
Skenario 5 slave 2 ambil alih	0	692	688	1.380	5.520	0,29%	16	67.959
Skenario 6 slave 1 ambil alih	696	692	692	2.080	8.320	0,38%	16	33.199
Skenario 7 master ambil alih	696	696	696	2.076	8.304	0%	12	27.777
Skenario 8 slave 1 dan slave 2 ambil alih	700	700	0	1.400	5.600	0%	12	30.867
Skenario 9 slave 1 dan slave 3 ambil alih	704	0	696	1.400	5.600	0,29%	4	32.395

Selanjutnya pada Pengujian kedua pada tanggal 30 april 2025. Node *master* berhasil mengirimkan 4.164 data dengan *packet loss* 0%, sementara setiap *slave* mengirimkan rata-

rata 696 data. *Slave 1* berperan sebagai *backup master* pada skenario 2, 6, 8, dan 9, serta sebagai *slave* biasa pada skenario 1 dan 7, dengan total 4.204 data dan *packet loss* 0%.

Slave 2 mengirimkan 4.888 data, menjadi *backup master* pada skenario 3 dan 5, serta sebagai *slave* biasa di skenario lain dengan *packet loss* sebesar 0,16%. Sementara itu, *Slave 3* mengirimkan total 5.592 data dan menjadi *backup master* pada skenario 4. Di skenario lain, ia bertindak sebagai *slave* biasa dengan *packet loss* sebesar 0,29%.

Berdasarkan Gambar 18–21, terlihat bahwa node *master* (Gambar 18) mampu menjaga kestabilan komunikasi tanpa *packet loss* sama sekali. Hal ini menandakan bahwa proses penerimaan dan pengiriman data dari seluruh *slave* ke *master* berlangsung konsisten tanpa gangguan. Pada *slave 1* (Gambar 19) dan *slave 2* (Gambar 20), grafik menunjukkan *packet loss* kecil yang muncul ketika keduanya beralih peran menjadi *backup master*. Meskipun demikian, nilai *packet loss* tetap di bawah 1% sehingga dampaknya tidak signifikan terhadap keandalan sistem. Sementara itu, *slave 3* (Gambar 21) mengalami penurunan data yang cukup tajam ketika transisi ke peran *backup master*, tetapi segera pulih kembali setelah proses *failover* selesai.

Secara keseluruhan, sistem berhasil mengirimkan total 18.848 data dengan *packet loss* rendah sebesar 0,13%. Menurut standar TIPHON, hal ini masuk dalam kategori “sangat bagus”. *Packet loss* umumnya terjadi saat transisi peran, ketika node pengganti baru aktif setelah node sebelumnya masih sempat

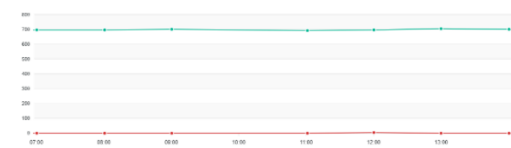
mengirimkan data. Namun, sistem kembali normal setelah proses *failover*, terbukti dengan pengiriman data yang konsisten. Adapun grafik *packet loss master* dan masing-masing *slave* bisa dilihat pada Gambar 18, 19, 20, dan 21.



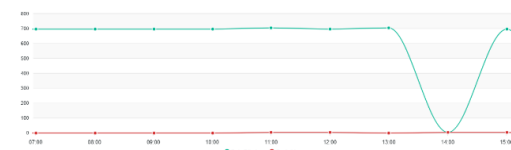
Gambar 18 Grafik *Packet loss* Pada Node *Master*



Gambar 19 Grafik *Packet loss* Pada Node *Slave* Satu



Gambar 20 Grafik *Packet loss* Pada Node *Slave* Dua



Gambar 21 Grafik *Packet loss* Pada Node *Slave* Tiga

Adapun Tabel 9 skenario untuk melihat hasil secara keseluruhan, bisa di liat pada tabel 3.

Tabel 3 Pengujian Sembilan Skenario

skenario	<i>Slave 1</i> (Jumlah data)	<i>Slave 2</i> (Jumlah data)	<i>Slave 3</i> (Jumlah data)	Jumlah Total data	Total byte data	Total packet loss	Data Terakhir Terkirim	waktu (ms)
Skenario 1 <i>master</i> ambil alih	696	696	696	2.088	8.352	0%	0	0
Skenario 2 <i>slave 1</i> ambil alih	700	696	696	2.092	8.368	0%	0	31.360
Skenario 3 <i>slave 2</i> ambil alih	0	700	696	1.396	5.584	0%	0	35.360
Skenario 4 <i>slave 3</i> ambil alih	0	0	696	696	2.784	0%	0	40.659
Skenario 5 <i>slave 2</i> ambil alih	0	692	688	1.380	5.520	0,29% (4 data)	16	67.179
Skenario 6 <i>slave 1</i> ambil alih	696	692	692	2.080	8.320	0,38% (8 data)	8	32.860
Skenario 7 <i>master</i> ambil alih	696	696	696	2.076	8.304	0%	24	23.433
Skenario 8 <i>slave 1</i> dan <i>slave 2</i>	700	700	0	1.400	5.600	0%	12	32.001

Tabel 3 (lanjutan)

skenario	Slave 1 (Jumlah data)	Slave 2 (Jumlah data)	Slave 3 (Jumlah data)	Jumlah Total data	Total byte data	Total packet loss	Data Terakhir Terkirim	waktu (ms)
Skenario 9 slave 1 dan slave 3 ambil alih	704	0	696	1.400	5.600	0,29% (4 data)	4	20.415

Selanjutnya, pada pengujian kedua tanggal 09 Juni 2025, node *master* berhasil mengirimkan 4.164 data dengan *packet loss* 0%, sementara setiap *slave* mengirimkan rata-rata 696 data. Slave 1 berperan sebagai *backup master* pada skenario 2, 6, 8, dan 9, serta sebagai *slave* biasa pada skenario 1 dan 7, dengan total 4.200 data dan *packet loss* 0%. Slave 2 mengirimkan 4.888 data, menjadi *backup master* pada skenario 3 dan 5, serta sebagai *slave* biasa di skenario lain dengan *packet loss* sebesar 0,16%. Sementara itu, *Slave 3* mengirimkan total 5.580 data dan menjadi *backup master* pada skenario 4, serta bertindak sebagai *slave* biasa di skenario lain dengan *packet loss* sebesar 0,29%. Hasil pengujian ini menunjukkan bahwa nilai *packet loss* diperoleh baik pada kondisi normal maupun saat dilakukan simulasi *failover* (perpindahan peran dari master mati ke *backup master*), bukan karena adanya gangguan jaringan eksternal.

Secara keseluruhan, sistem berhasil mengirimkan total 18.832 data dengan *packet loss* rendah sebesar 0,13%. Menurut standar TIPHON, hal ini masuk dalam kategori “sangat bagus”. *Packet loss* umumnya terjadi saat transisi peran, ketika node pengganti baru aktif setelah node sebelumnya masih sempat mengirimkan data. Namun, sistem kembali normal setelah proses *failover*, terbukti dengan

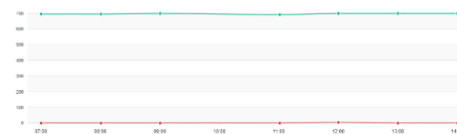
pengiriman data yang konsisten. Adapun grafik *packet loss* master dan masing-masing *slave* bisa dilihat pada Gambar 22, 23, 24, dan 25.



Gambar 22 Grafik *Packet loss* Pada Node Master



Gambar 23 Grafik *Packet loss* Pada Node Slave Satu



Gambar 24 Grafik *Packet loss* Pada Node Slave Dua



Gambar 25 Grafik *Packet loss* Pada Node Slave Tiga

Adapun Tabel 9 skenario untuk melihat hasil secara keseluruhan, bisa dilihat pada tabel 4 .

Tabel 4 Pengujian Sembilan Skenario

skenario	Slave 1 (Jumlah data)	Slave 2 (Jumlah data)	Slave 3 (Jumlah data)	Jumlah Total data	Total byte data	Total packet loss	Data Terakhir Terkirim	waktu (ms)
Skenario 1 master ambil alih	696	696	696	2.088	8.352	0%	0	0
Skenario 2 slave 1 ambil alih	700	696	696	2.092	8.368	0%	0	31.659
Skenario 3 slave 2 ambil alih	0	700	696	1.396	5.584	0%	0	35.415
Skenario 4 slave 3 ambil alih	0	0	692	692	2.768	0%	0	40.637
Skenario 5 slave 2 ambil alih	0	692	688	1.392	5.568	0,29% (4 data)	16	66.696

Tabel 4 (Lanjutan)

skenario	Slave 1 (Jumlah data)	Slave 2 (Jumlah data)	Slave 3 (Jumlah data)	Jumlah Total data	Total byte data	Total packet loss	Data Terakhir Terkirim	waktu (ms)
Skenario 6 slave 1 ambil alih	696	692	692	2.088	8.352	0,38%(8 data)	8	32.997
Skenario 7 master ambil alih	696	696	696	2.076	8.304	0%	24	20.953
Skenario 8 slave 1 dan slave 2	700	700	0	1.400	5.600	0%	4	31.949
Skenario 9 slave 1 dan slave 3 ambil alih	704	0	696	1.400	5.600	0,29% (4 data)	4	20.624

Secara keseluruhan, sistem monitoring tanah berbasis topologi mesh menunjukkan kinerja andal dengan kemampuan failover otomatis dan komunikasi yang stabil. Mekanisme hirarki (*master* → *slave 1* → *slave 2* → *slave 3*) memungkinkan transisi peran antar-node dalam 20–67 detik, didukung oleh heartbeat 20 detik dan acknowledgment untuk deteksi kegagalan yang responsif. Meskipun terjadi overlap data dan variasi ukuran paket (2.784–8.400 byte), pengiriman tetap stabil dengan packet loss rendah (0,13%), sesuai standar TIPHON. Topologi mesh memberikan ketahanan jaringan dengan fitur self-healing, sementara dashboard web memudahkan pemantauan data secara real-time. Sistem ini berhasil mengatasi tantangan komunikasi WSN dan menjaga kontinuitas pengiriman data meskipun terjadi kegagalan node.

4. KESIMPULAN

Penelitian ini membuktikan bahwa sistem monitoring tanah berbasis master-slave dengan topologi mesh mampu menjaga keandalan komunikasi dengan packet loss sangat rendah sebesar 0,13% pada setiap pengujian (18.852 data pada pengujian 1, 18.848 data pada pengujian 2, dan 18.832 data pada pengujian 3), sehingga memenuhi standar TIPHON kategori “sangat bagus”. Topologi mesh dan mekanisme failover bertingkat (*master* → *slave 1* → *slave 2* → *slave 3*) memungkinkan jalur alternatif saat terjadi kegagalan node, dengan protokol heartbeat, request data, dan acknowledgment (ACK) sebagai deteksi kegagalan dan transisi peran, menghasilkan waktu failover tercepat 20,415 detik dan terlama 67,959 detik. Packet loss yang terjadi bersifat sementara akibat keterlambatan respons saat transisi, dan sistem kembali stabil setelahnya. Untuk pengembangan lebih lanjut, disarankan memperluas kapasitas jaringan dengan penambahan node master agar

tidak seluruh beban dialihkan ke node slave, serta membangun ekosistem digital terintegrasi yang tidak hanya melakukan monitoring kelembaban tanah, tetapi juga mampu menjalankan aksi otomatis seperti irigasi pintar, rekomendasi pupuk, maupun penyiraman otomatis.

DAFTAR PUSTAKA

- [1] D. P. Sari, N. Kurniasih, And M. T. B. Sitorus, “Prototype Alat Monitoring Suhu, Kelembaban Dan Kecepatan Angin Untuk Smart Farming Menggunakan Komunikasi Lora Dengan Daya Listrik Menggunakan Panel Surya,” *Kilat*, Vol. 10, No. 2, Pp. 370–380, Oct. 2021, Doi: 10.33322/Kilat.V10i2.1376.
- [2] W. Dwi Meilianto, W. Indrasari, And E. Budi, “Prosiding Seminar Nasional Fisika (E-Journal) Snf2022 Seminar Nasional Fisika 2021 Program Studi Fisika Dan Pendidikan Fisika, Fakultas Mipa,” *Prosiding Seminar Nasional Fisika (E-Journal)*, Doi: 10.21009/03.Snf2022.
- [3] K. A. Gunawan, “Rancang Bangun Alat Pengukur Suhu Tanah Sebagai Alat Bantu Penentu Benih Sayuran Yang Akan Dibudidayakan,” Unnes, Semarang, 2015.
- [4] C. Wijaya, A. Pratama, And P. Putra, “Pembangunan Aplikasi Sharing Internet Menggunakan Karakteristik Mesh Network Pada Ponsel Cerdas Berbasis Android,” *Jurnal Ilmiah Teknologi Informasi Terapan*, Vol. Iii, No. 3, 2017, Doi: <https://doi.org/10.33197/Jitter.Vol3.Iss3.2017.135>.
- [5] J. P. Dias, B. Lima, J. P. Faria, A. Restivo, And H. S. Ferreira, “Visual Self-Healing Modelling For Reliable Internet-Of-Things Systems,” *Lecture Notes In Computer Science (Including Subseries Lecture Notes In Artificial Intelligence And Lecture Notes In Bioinformatics)*, Vol. 12141 Lncs, Pp. 357–370, 2020, Doi:



- https://Link.Springer.Com/Chapter/10.1007/978-3-030-50426-7_27.
- [6] M. Putri Anastasia, D. Nur Ilham, And F. Atabiq, "Development Of Nrf24l01 Communication Module-Based Temperature Monitoring System," *Jrcs*, Vol. 2, No. 1, Pp. 2770–1800, 2022, [Online]. Available: [Http://Journal.Station-It.Org/Index.Php/Jrcs](http://Journal.Station-It.Org/Index.Php/Jrcs)
- [7] W. I. S. A. Talli, J. D. Irawan, And F. X. Ariwibisono, "Rancang Bangun Sistem Monitoring Kualitas Tanah Untuk Tanaman Cabai Berbasis Iot (Internet Of Things)," *Jati (Jurnal Mahasiswa Teknik Informatika)*, 2023. Doi: <https://doi.org/10.36040/Jati.V7i4.7540>.
- [8] Espressif, "Espressif Esp32." Accessed: Mar. 24, 2024. [Online]. Available: <https://www.espressif.com/en/products/socs/esp32>
- [9] Algorista, "Sensor Kelembaban Tanah Atau Soil Moisture." Accessed: Mar. 24, 2024. [Online]. Available: <https://www.algorista.com/2020/01/sensor-soil-moisture.html>
- [10] A. Wajiansyah, Supriadi, N. Ramadhan, R. Sandria, And M. Dian Pratama, "Implementasi Master-Slave Pada Embedded System," *Elkha*, 2020, Doi: <https://doi.org/10.26418/Elkha.V12i1>.
- [11] Aorinka Anendya, "Topologi Mesh." Accessed: Mar. 05, 2025. [Online]. Available: <https://www.dewaweb.com/blog/mengenal-topologi-mesh/>
- [12] M. D. Khairy, A. N. Defitri, A. Hadi, A. Nuzuli, A. L. Dyakiyyah, And A. Heryanto, "Netplg Journal Of Network And Computer Applications Pengolahan Data Sensor Iot Dengan Apache Spark Menggunakan Metode Batch Processing," 2023, [Online]. Available: <https://jurnal.netplg.com/jnca>
- [13] Aws, "Server Cloud." Accessed: Mar. 30, 2024. [Online]. Available: <https://aws.amazon.com/id/what-is/cloud-server/#:~:text=Server%20cloud%20adalah%20server%20virtual,Memelihara%20server%20fisik%20mereka%20sendiri>.
- [14] Kamal, Firdayanti, U. Mahanin Tyas, A. Apri Buckhari, And Pattasang, "Implementasi Aplikasi Arduino Ide Pada Mata Kuliah Sistem Digital," *Teknos*, Vol. 1, No. 1, 2023, Doi: <https://doi.org/10.59638/Teknos.V1i1.40>.
- [15] F. Natacia And E. Mailoa, "Perancangan Aplikasi Jaringan Dokumentasi Dan Informasi Hukum Berbasis Web Menggunakan Framework Laravel," *Jurnal Pendidikan Tambusai (Jpt)*, Vol. 6, Feb. 2022, Doi: <https://doi.org/10.31004/Jptam.V6i1.3172>.
- [16] Wireshark, "Wireshark." Accessed: Apr. 19, 2025. [Online]. Available: https://www.wireshark.org/docs/wsug_html_chunked/ChapterIntroduction.html
- [17] Tiphon, "Telecommunications And Internet Protocol Harmonization Over Networks (Tiphon) Release 3; End-To-End Quality Of Service In Tiphon Systems; Part 7: Design Guide For Elements Of A Tiphon Connection From An End-To-End Speech Transmission Performance Point Of View," 2002.